ARCHITECTURAL DESIGN

How a software system is organized and designed.

Advantages of Having an Architectural Design

Stakeholder Communication

- A high-level presentation may be used as a focus for discussions with different stakeholders.

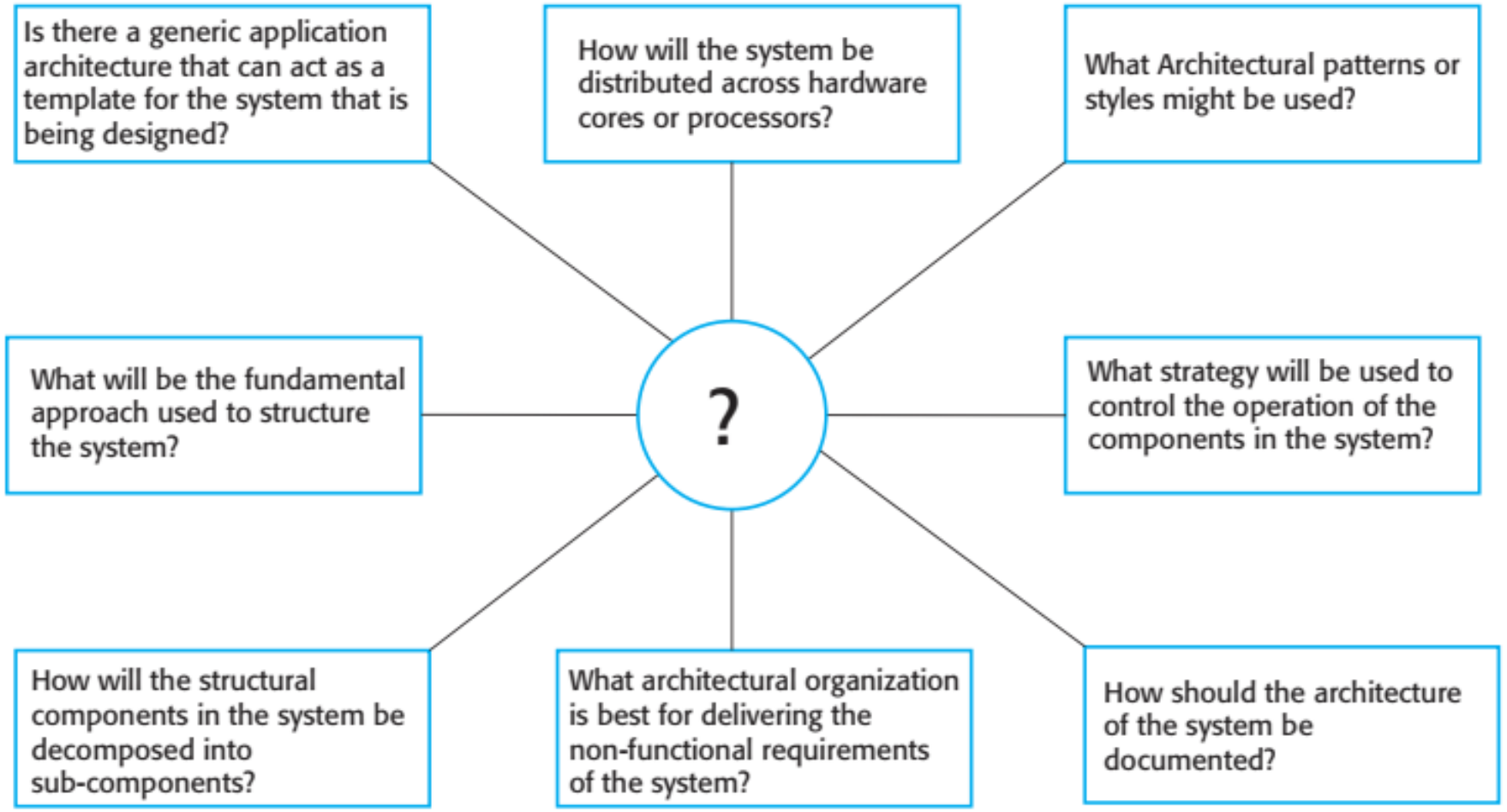
System analysis

- Developed at an early stage requires analysis which have a profound effect on whether or not the system can meet critical requirements (performance, reliability, and maintainability).

Large-scale Reuse

- System architecture is often the same for systems with similar requirement and so can support large-scale software reuse.

Architectural Design Decisions



The choice of software architecture depends on the non-functional requirement of the system:

PERFORMANCE

- Design should localize critical operations within a small number of components, with these components deployed on the same computer rather than distributed across the network.
- Use relatively large components rather than small, finer-grade components
- Using large components reduces the number of component communications as most of the interactions related system features take place within a component.
- Allow system to be replicated and executed on different processors.

The choice of software architecture depends on the non-functional requirement of the system:

SECURITY

- A layered structure for the architecture should be used, with the most critical assets protected in the innermost layers and a high level of security validation applied to these layers.

The choice of software architecture depends on the non-functional requirement of the system:

SAFETY

- Design safety-related operations are co-located in a single component or in a small number of components.
- Design may reduce the costs and problems of safety validation and may make it possible to provide related protection systems that can safely shut down the system in the event of failure.

The choice of software architecture depends on the non-functional requirement of the system:

AVAILABILITY

- Designed to include redundant components so that it is possible to replace and update components without stopping the system.

The choice of software architecture depends on the non-functional requirement of the system:

MAINTAINABILITY

- Designed using fine-grain, self-contained components that may readily be changed.
- Producers of data should be separate from consumers, and shared data structures should be avoided.

Architectural Views

Logical View: shows the key abstractions in the system as objects or object classes.

Process View: show how at runtime, the system is composed of interacting processes. Useful for making judgments about non-functional system characteristics such as performance and availability.

Development View: show how the software is decomposed for development. It shows the breakdown of the software into components that are implemented by a single developer or development team. This is useful for software managers and programmers.

Architectural Views

Physical View: shows the system hardware and how software components are distributed across the processors in the system. This is useful for system engineers planning a system deployment.

Conceptual View: show the abstract view of the system that can be the basis for decomposing high-level requirements into more detailed specifications, help engineers make decisions about components that can be reused, and represent a product line.

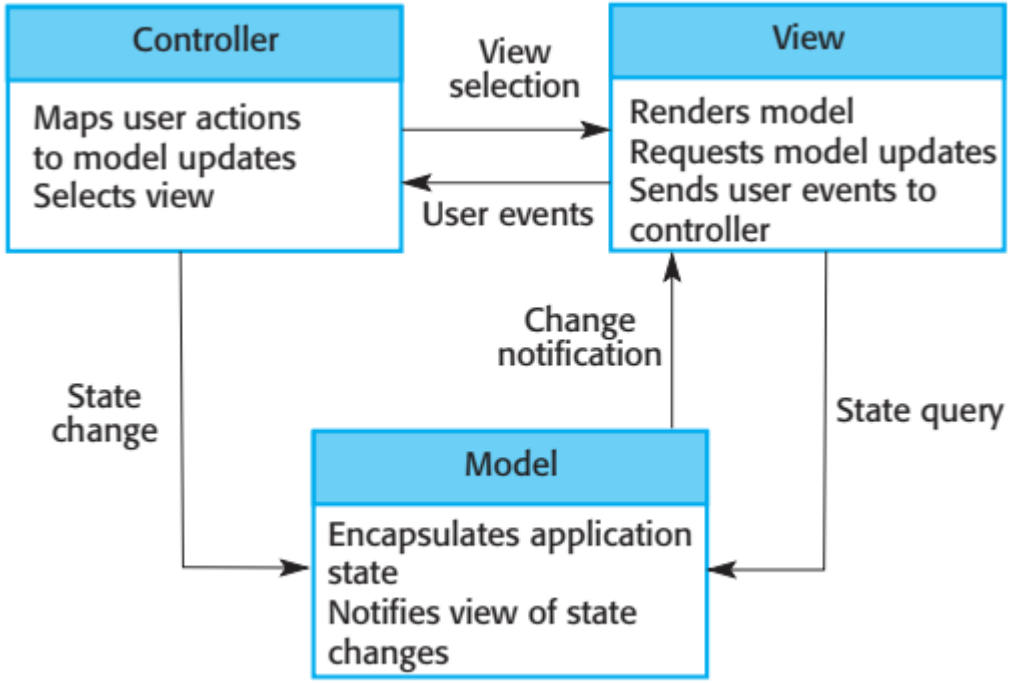
Architectural Patterns

- MVC (Model-View-Controller)
- Layered Architecture
- Repository Architecture
- Client-Server Architecture
- Pipe and Filter Architecture
- Application Architecture
  - Transaction Processing Systems
  - Information Systems
  - Language Process System

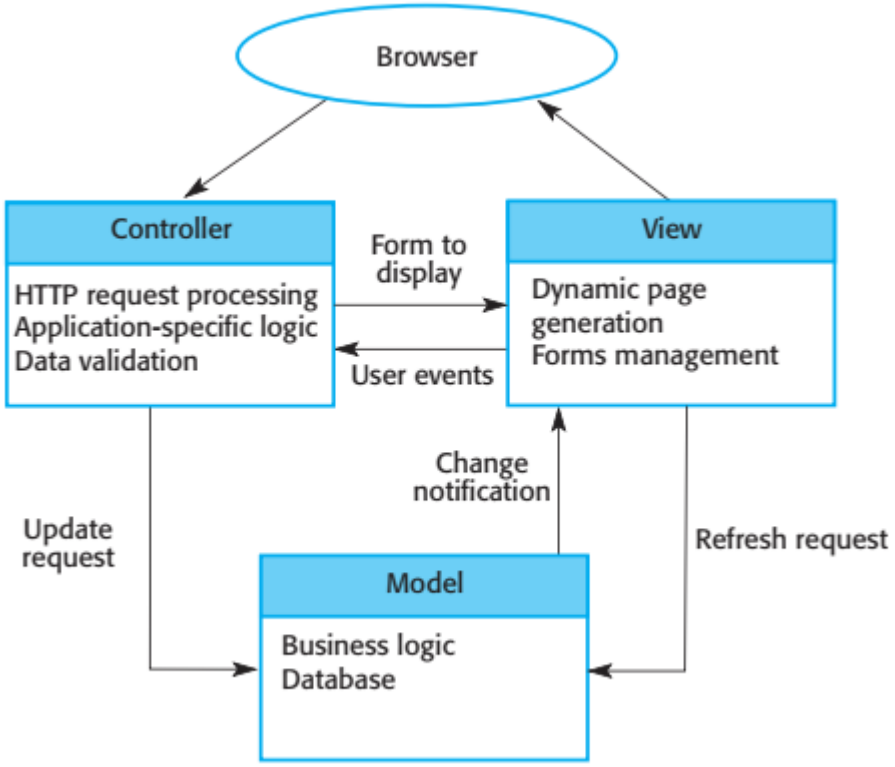
MVC (Model-View-Controller)

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.5.
Example	Figure 6.6 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways, with changes made in one representation shown in all of them.
Disadvantages	May involve additional code and code complexity when the data model and interactions are simple.

MVC (Model-View-Controller)



Organization of the Model-View-Controller

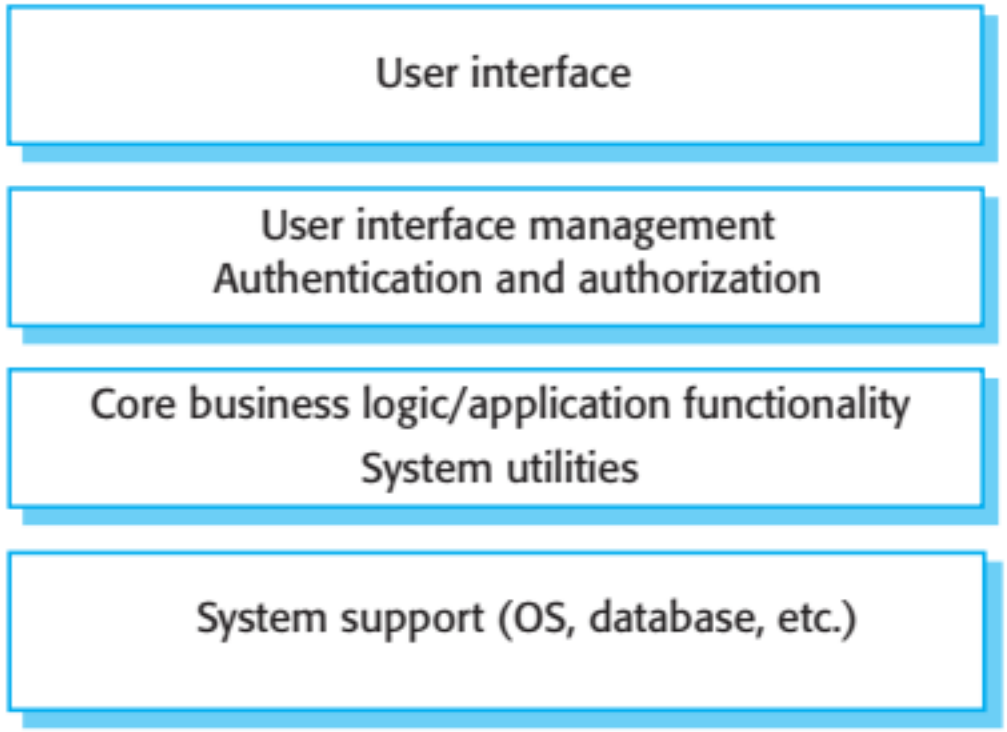


Web Application Architecture Using the MVC Pattern

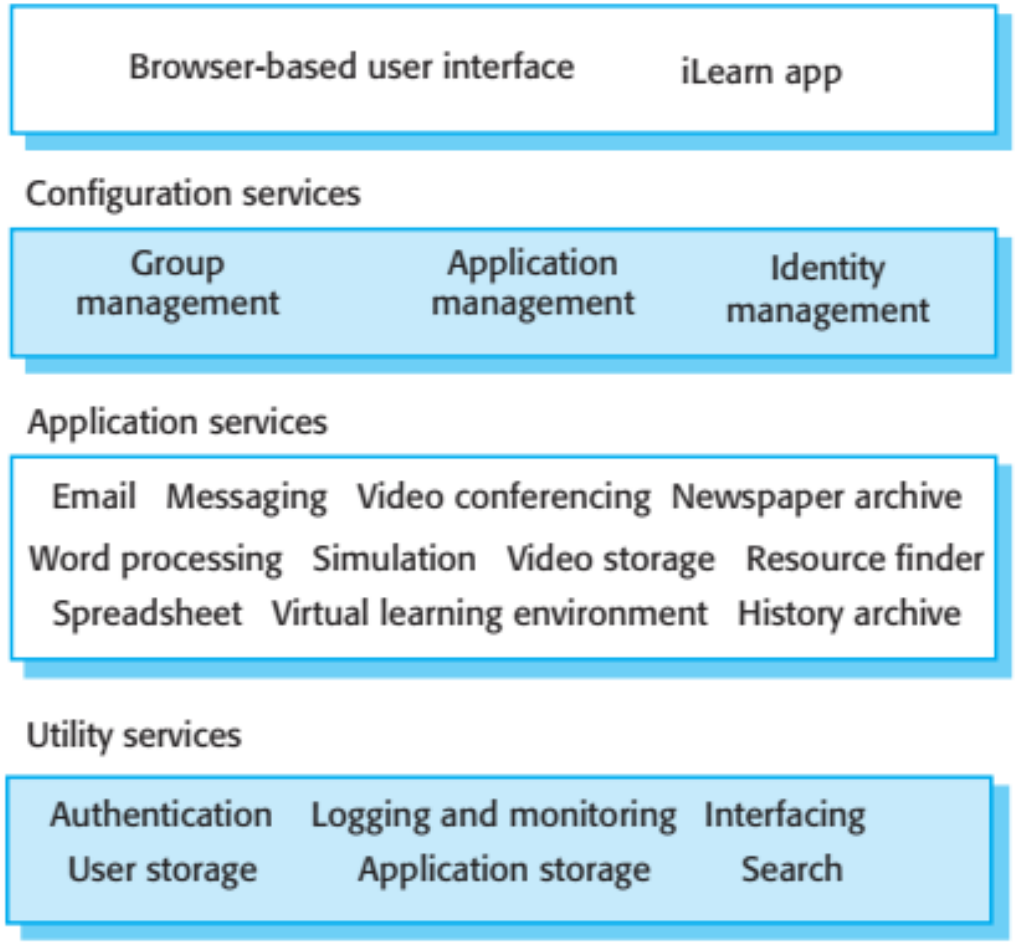
Layered Architecture

Name	Layered architecture
Description	Organizes the system into layers, with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest level layers represent core services that are likely to be used throughout the system. See Figure 6.8.
Example	A layered model of a digital learning system to support learning of all subjects in schools (Figure 6.9).
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multilevel security.
Advantages	Allows replacement of entire layers as long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Layered Architecture



A generic layered architecture

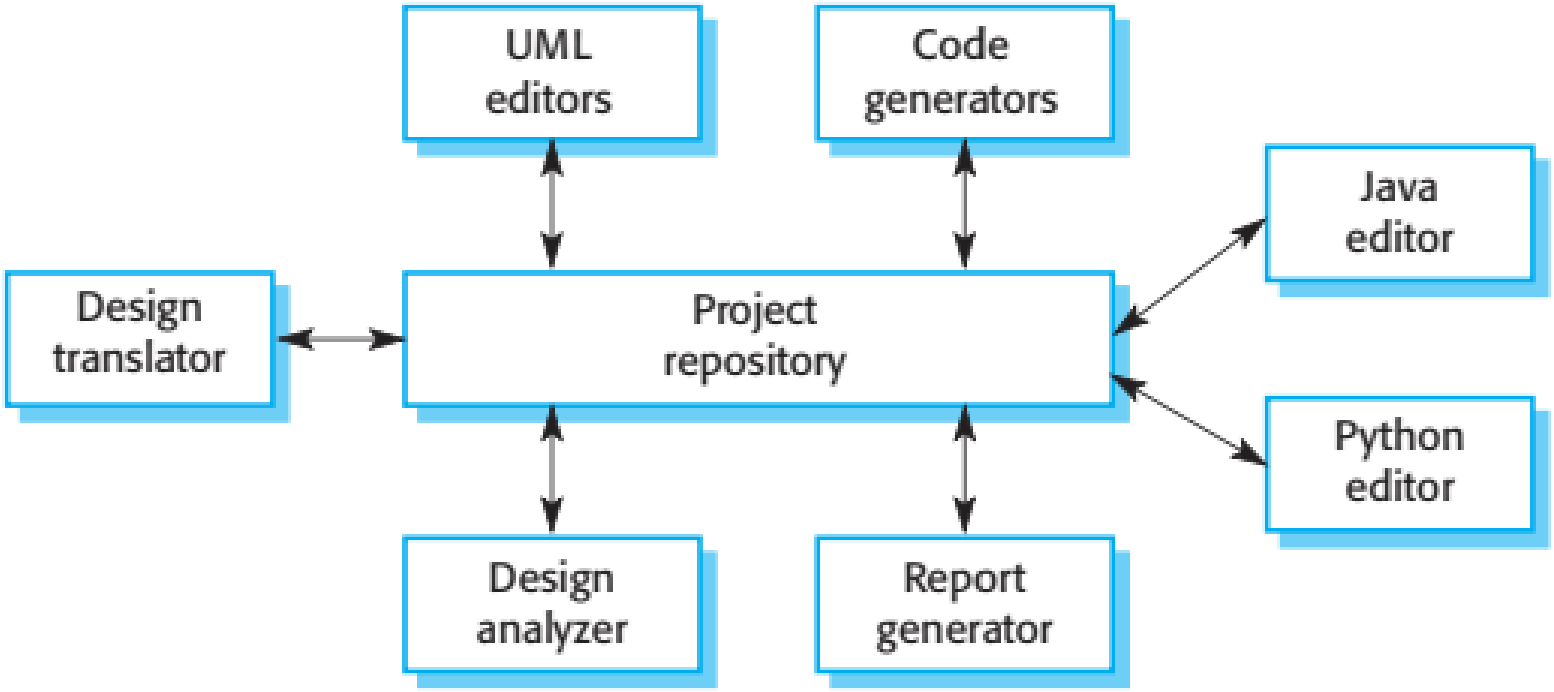


Layered architecture of the iLearn system

Repository Architecture

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.11 is an example of an IDE where the components use a repository of system design information. Each software tool generates information, which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent; they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

Repository Architecture

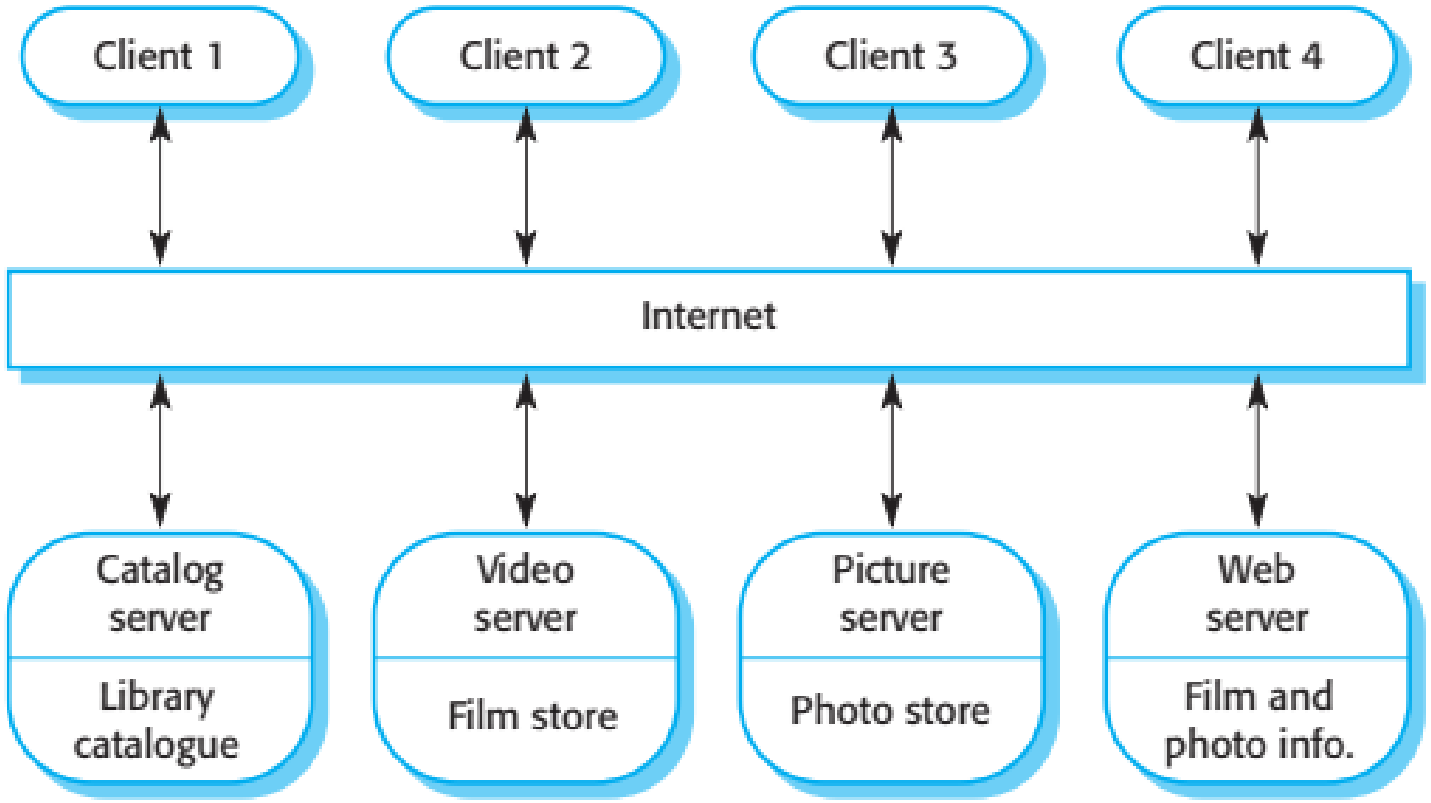


A repository architecture for an IDE

Client-Server Architecture

Name	Client–server
Description	In a client–server architecture, the system is presented as a set of services, with each service delivered by a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.13 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure and so is susceptible to denial-of-service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. Management problems may arise if servers are owned by different organizations.

Client-Server Architecture

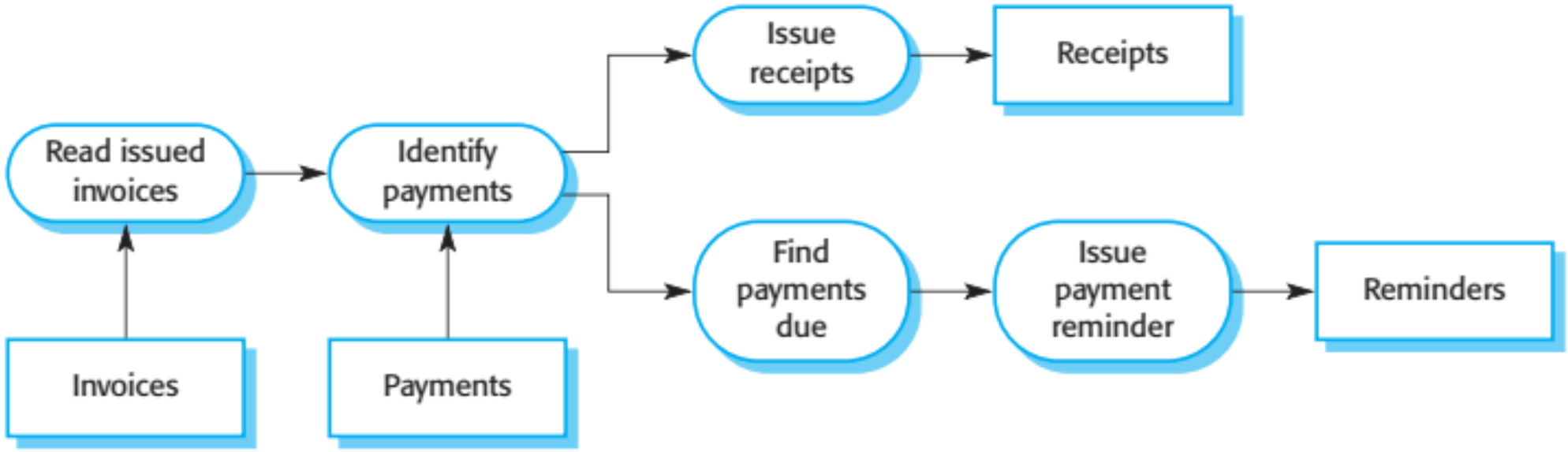


A client-server architecture for a film library

Pipe and Filter Architecture

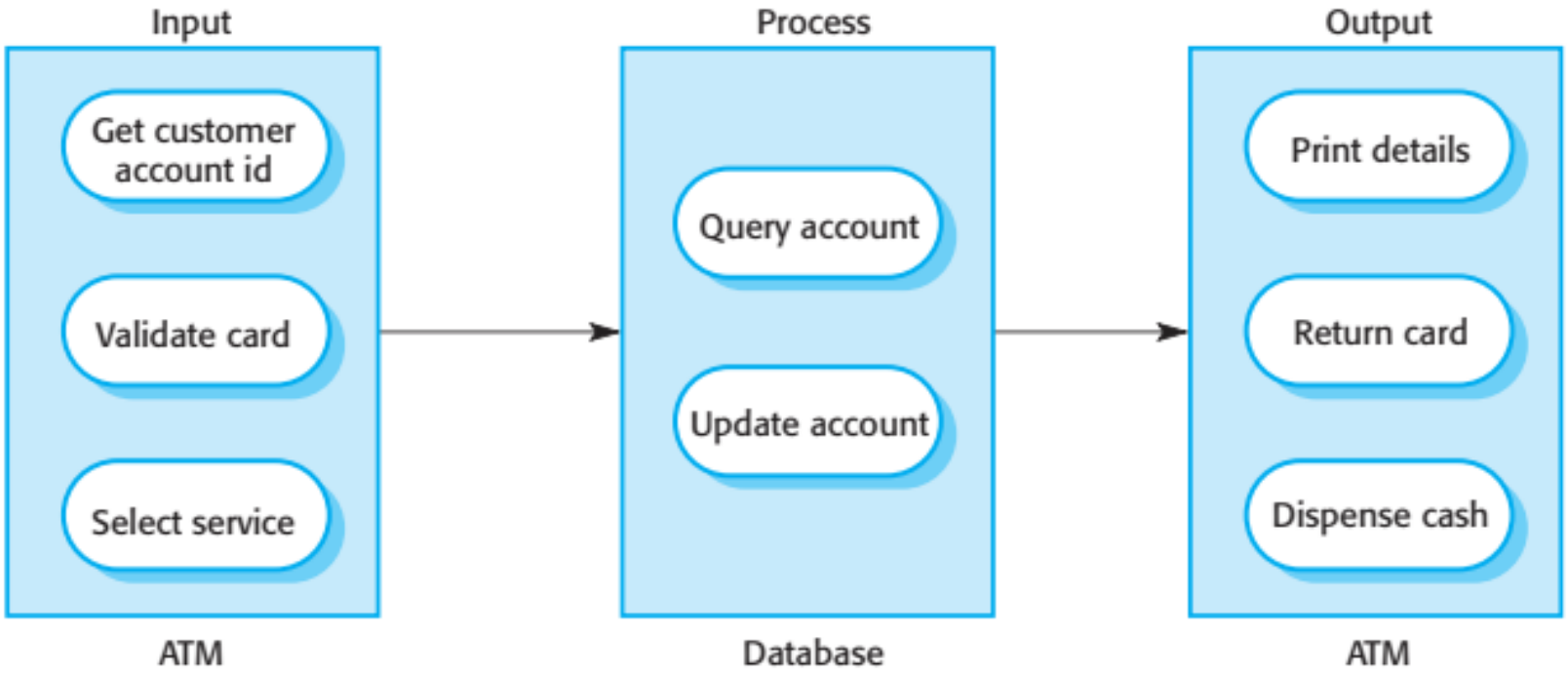
Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.15 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data-processing applications (both batch and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse architectural components that use incompatible data structures.

Pipe and Filter Architecture



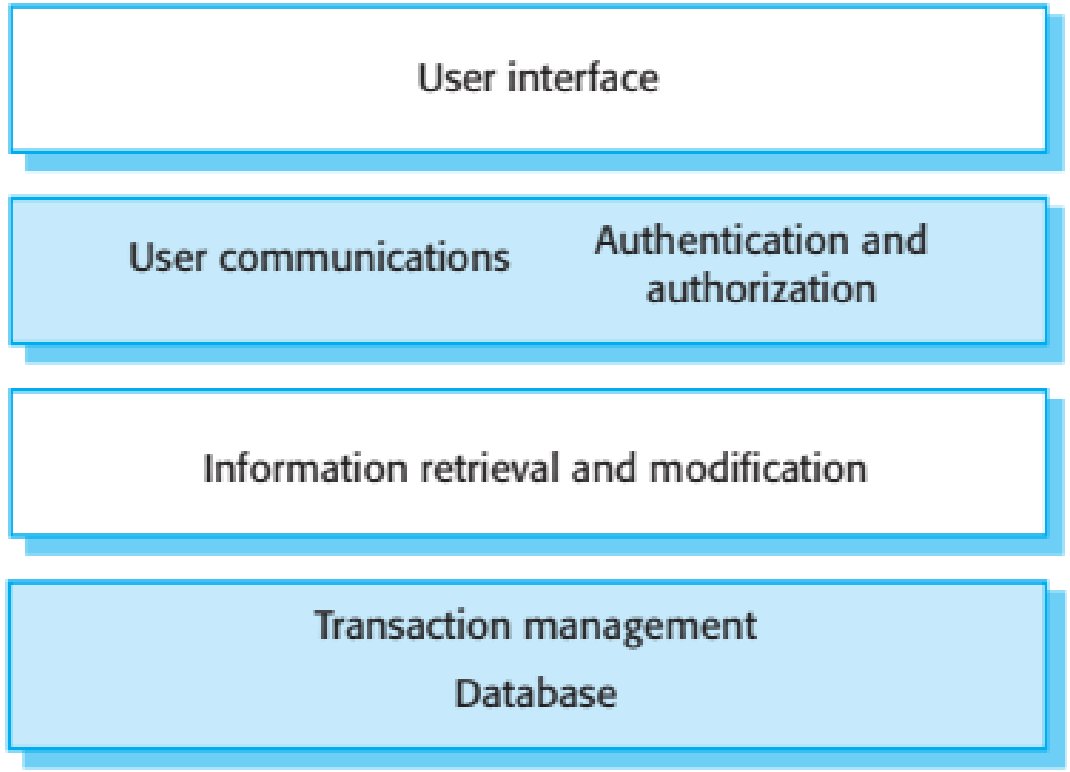
An example of the pipe and filter architecture

Application Architecture: TPS

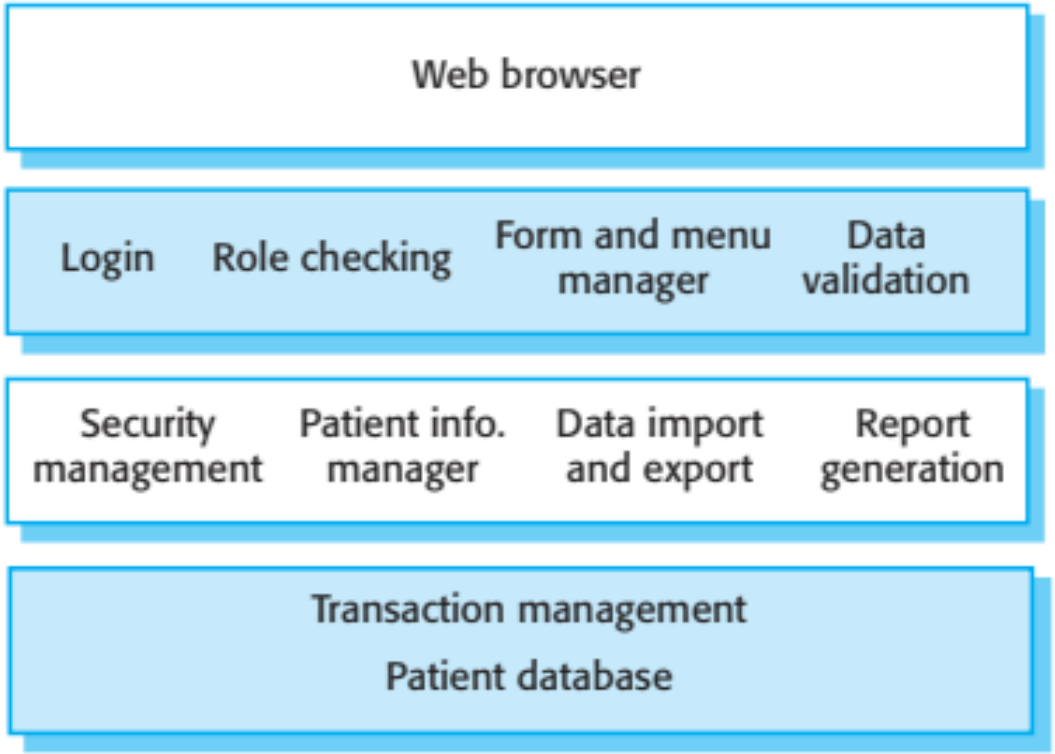


Software Architecture of an ATM Machine

Application Architecture: IS

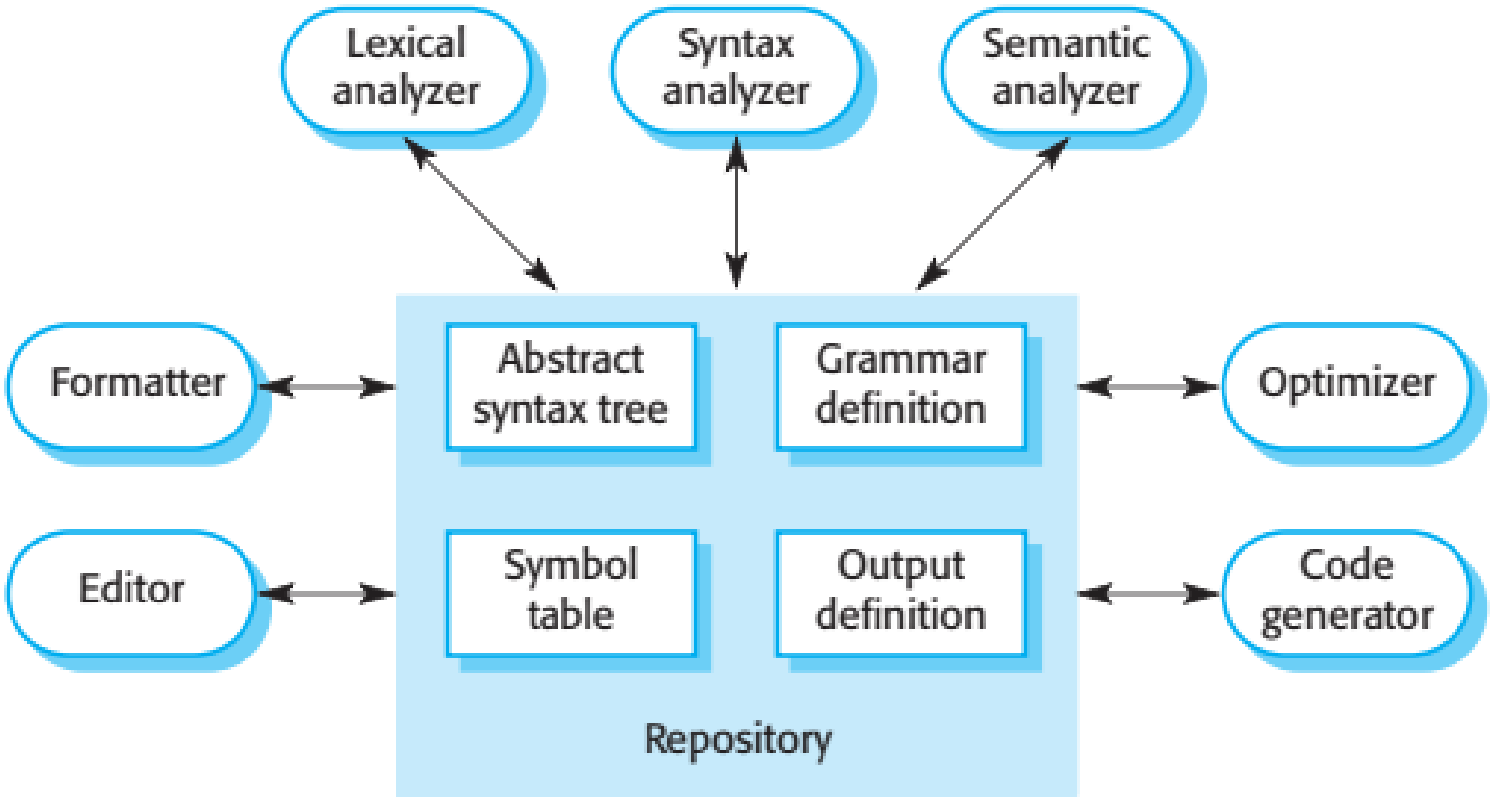


Layered Information System Architecture



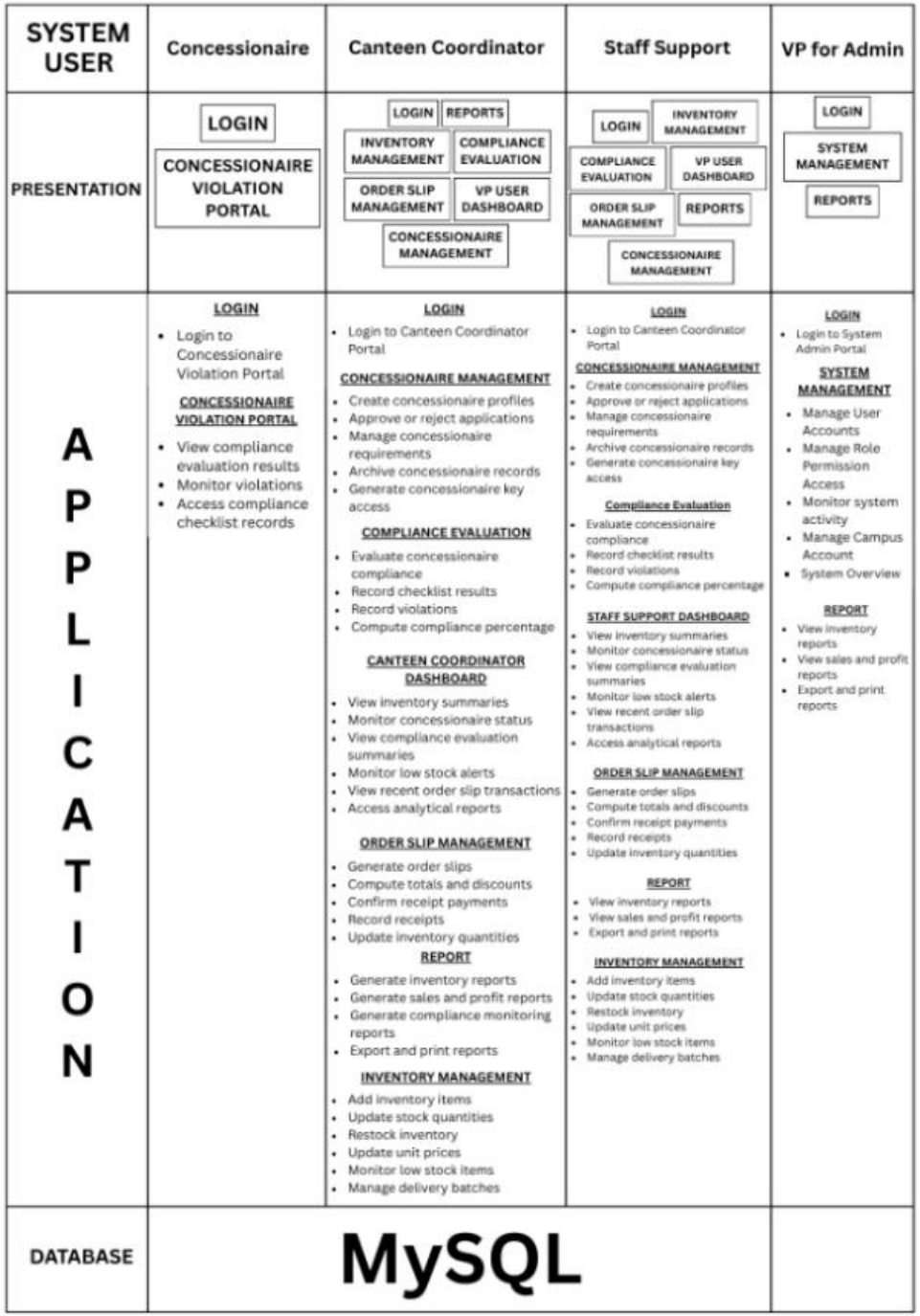
Architecture of the Mentcare System

Application Architecture: LPS



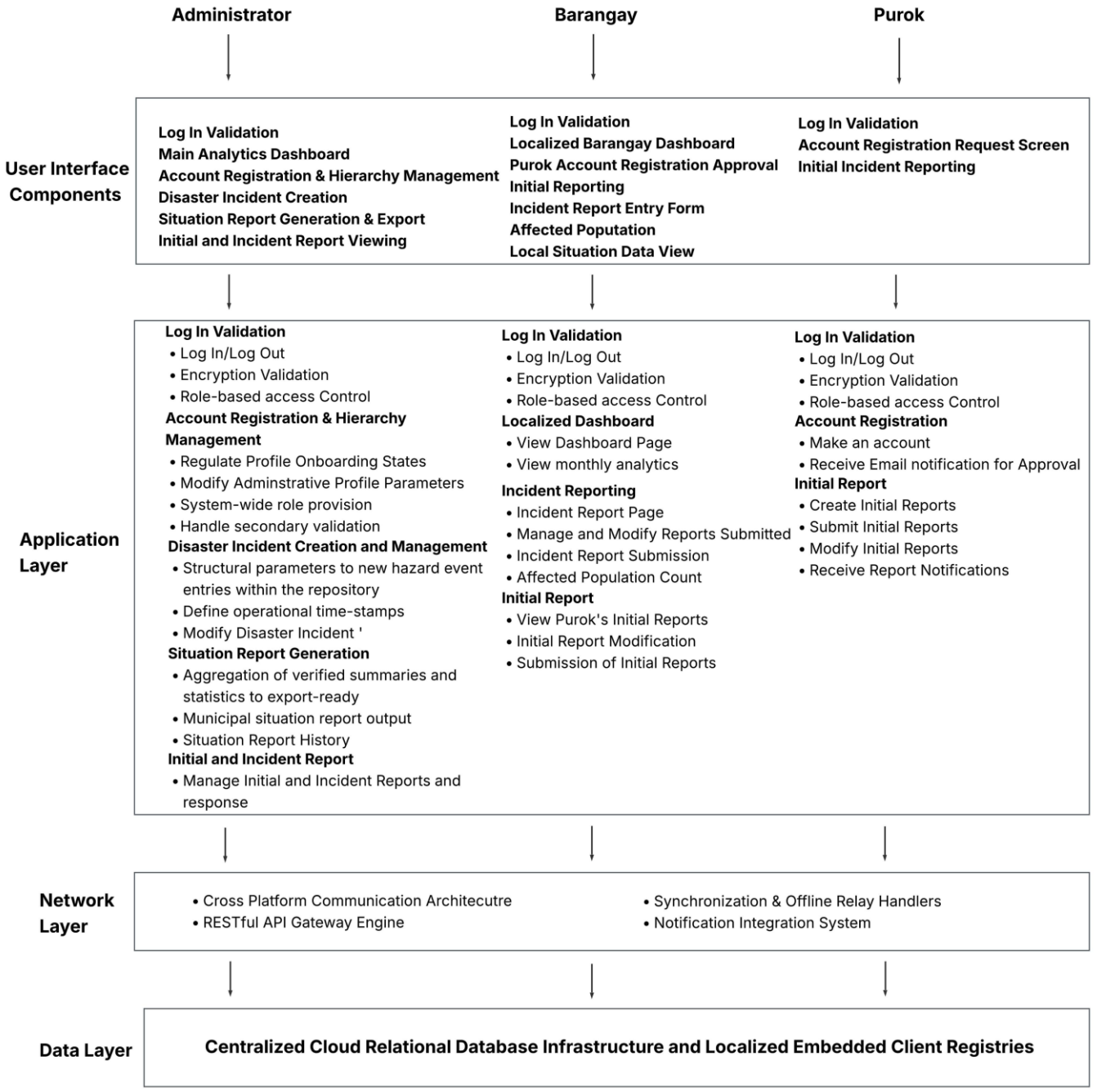
Architecture of a Language Processing System

Sample Architectural Design in Capstone Projects

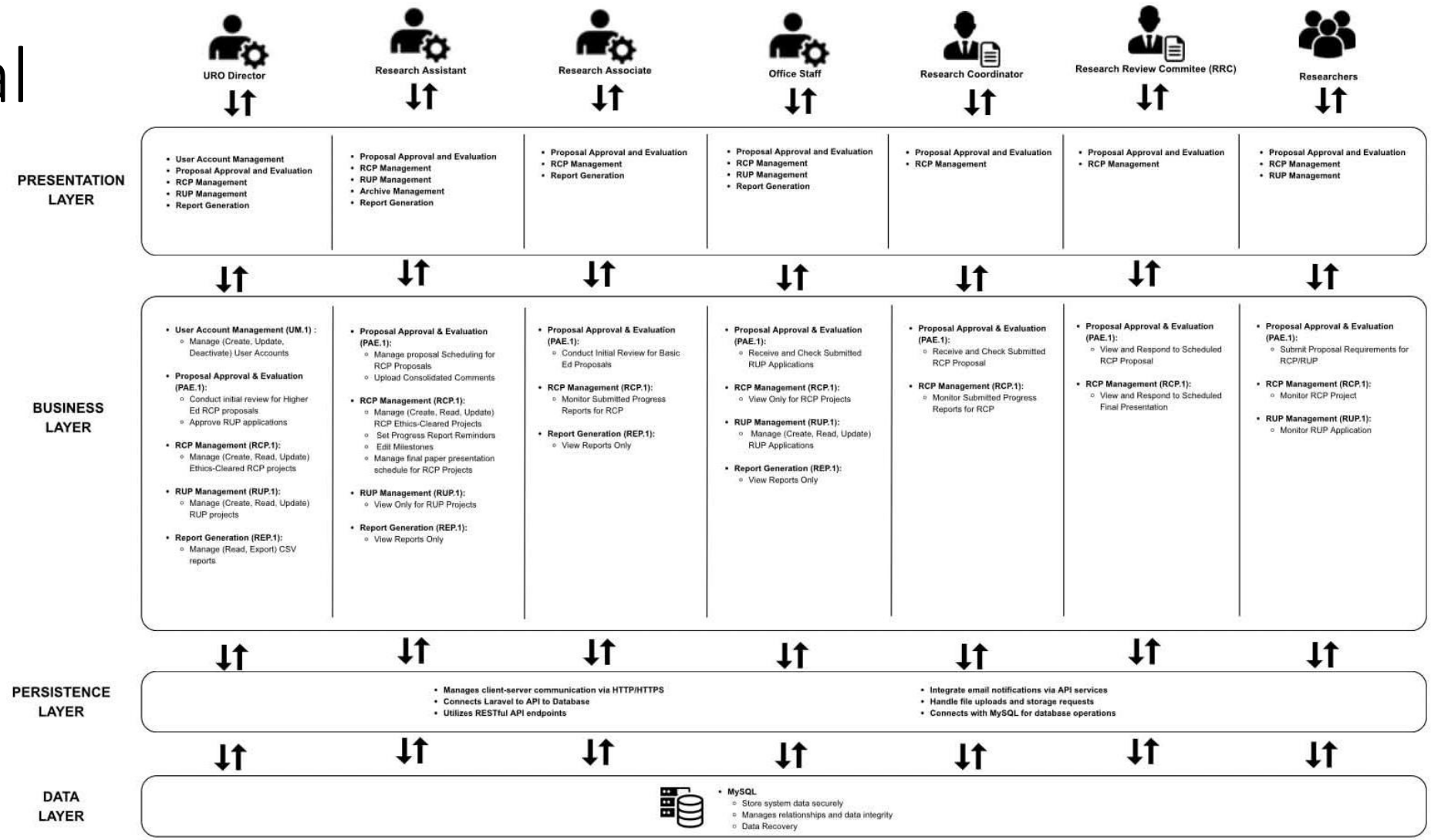


SYSTEM USER	Concessionaire	Canteen Coordinator	Staff Support	VP for Admin
PRESENTATION	LOGIN CONCESSIONAIRE VIOLATION PORTAL	LOGIN REPORTS INVENTORY MANAGEMENT COMPLIANCE EVALUATION ORDER SLIP MANAGEMENT VP USER DASHBOARD CONCESSIONAIRE MANAGEMENT	LOGIN INVENTORY MANAGEMENT COMPLIANCE EVALUATION VP USER DASHBOARD ORDER SLIP MANAGEMENT REPORTS CONCESSIONAIRE MANAGEMENT	LOGIN SYSTEM MANAGEMENT REPORTS
APPLICATION	**LOGIN** • Login to Concessionaire Violation Portal **CONCESSIONAIRE VIOLATION PORTAL** • View compliance evaluation results • Monitor violations • Access compliance checklist records	**LOGIN** • Login to Canteen Coordinator Portal **CONCESSIONAIRE MANAGEMENT** • Create concessionaire profiles • Approve or reject applications • Manage concessionaire requirements • Archive concessionaire records • Generate concessionaire key access **COMPLIANCE EVALUATION** • Evaluate concessionaire compliance • Record checklist results • Record violations • Compute compliance percentage **CANTEEN COORDINATOR DASHBOARD** • View inventory summaries • Monitor concessionaire status • View compliance evaluation summaries • Monitor low stock alerts • View recent order slip transactions • Access analytical reports **ORDER SLIP MANAGEMENT** • Generate order slips • Compute totals and discounts • Confirm receipt payments • Record receipts • Update inventory quantities **REPORT** • Generate inventory reports • Generate sales and profit reports • Generate compliance monitoring reports • Export and print reports **INVENTORY MANAGEMENT** • Add inventory items • Update stock quantities • Restock inventory • Update unit prices • Monitor low stock items • Manage delivery batches	**LOGIN** • Login to Canteen Coordinator Portal **CONCESSIONAIRE MANAGEMENT** • Create concessionaire profiles • Approve or reject applications • Manage concessionaire requirements • Archive concessionaire records • Generate concessionaire key access **Compliance Evaluation** • Evaluate concessionaire compliance • Record checklist results • Record violations • Compute compliance percentage **STAFF SUPPORT DASHBOARD** • View inventory summaries • Monitor concessionaire status • View compliance evaluation summaries • Monitor low stock alerts • View recent order slip transactions • Access analytical reports **ORDER SLIP MANAGEMENT** • Generate order slips • Compute totals and discounts • Confirm receipt payments • Record receipts • Update inventory quantities **REPORT** • View inventory reports • View sales and profit reports • Export and print reports **INVENTORY MANAGEMENT** • Add inventory items • Update stock quantities • Restock inventory • Update unit prices • Monitor low stock items • Manage delivery batches	**LOGIN** • Login to System Admin Portal **SYSTEM MANAGEMENT** • Manage User Accounts • Manage Role Permission Access • Monitor system activity • Manage Campus Account • System Overview **REPORT** • View inventory reports • View sales and profit reports • Export and print reports
DATABASE	MySQL			

Sample Architectural Design in Capstone Projects



Sample Architectural Design in Capstone Projects



Sample Architectural Design in Capstone Projects

